

What is Agentic AI?

Traditional AI vs. Agentic AI

A traditional AI system is essentially a very well-read encyclopedia. It can answer questions from what it learned during training, but it cannot reach into the world to fetch fresh data or take an action on your behalf:

```
User: "What is the weather like in New York?"
Traditional AI: "I can't provide live weather information. Weather refers to
                atmospheric conditions including temperature, humidity..."
```

An agentic system can plan and act. Given the same prompt, it decides what it needs to do, calls the right service, processes what comes back, and returns a real answer:

```
User: "What's the weather forecast for New York this weekend?"
Agentic AI:
1. Problem: User wants a NYC weekend forecast
2. Plan:    Resolve NYC coordinates and call the National Weather Service
3. Action:  Calls the NWS Points API, then the Forecast API
4. Result:  "This weekend in New York: Saturday sunny with highs of 75°F,
            Sunday partly cloudy with a 20% chance of rain..."
```

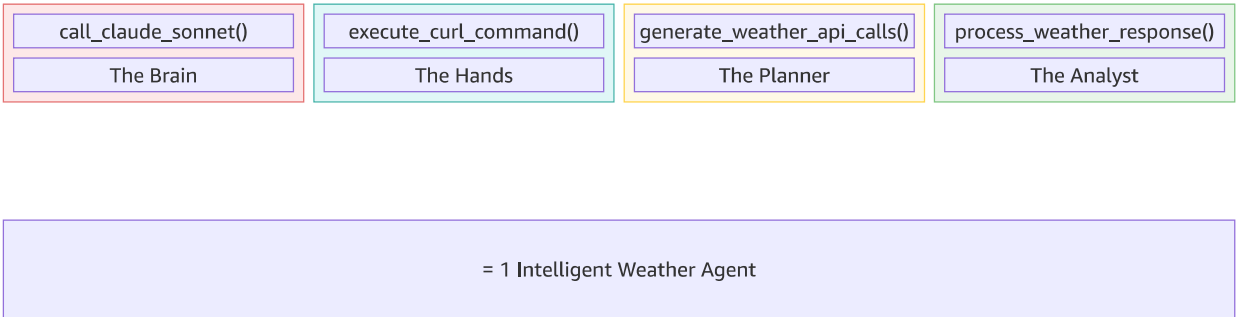
That four-phase loop — **Problem** → **Plan** → **Action** → **Result** — is the spine of every agent in this workshop.

Phase	What happens	In our weather agent
Problem	The user states a goal in natural language	"What's the weather in Seattle?"
Plan	The model decides what needs to happen	Claude generates the right NWS API URL

Phase	What happens	In our weather agent
Action	The agent executes against the real world	<code>curl</code> calls the Points API and the Forecast API
Result	The model turns raw output into a useful answer	Claude turns the JSON into a readable summary

The Four Building Blocks

Our weather agent is four Python functions, each playing a distinct role. We will write all four in section 4.



Function	Role	What it does
<code>call_claude_sonnet()</code>	The Brain	Sends prompts to Claude 4.5 Sonnet on Amazon Bedrock
<code>generate_weather_api_calls()</code>	The Planner	Asks Claude to produce the correct NWS API URL for a location
<code>execute_curl_command()</code>	The Hands	Makes the actual HTTP request to the outside world
<code>process_weather_response()</code>	The Analyst	Asks Claude to turn raw JSON into a readable forecast

Four functions. No frameworks. Pure Python and `boto3`.

Why Amazon Bedrock

Amazon Bedrock gives us managed access to Claude 4.5 Sonnet over a single `boto3` call, with the same authentication and region setup we use for every other AWS

service. That keeps the workshop focused on the agent pattern rather than on model hosting, infrastructure, or per-vendor SDKs.

Claude 4.5 Sonnet specifically suits this workload because it handles two things well: reasoning about a fuzzy input (a location description) to produce a precise output (a URL), and reading a noisy JSON blob to produce a clean summary. Both are core to the planner and analyst roles above.

Once your environment is set up, we will look at how the four building blocks chain together end to end.

[Previous](#)[Next](#)

© 2008 - 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved.

[Privacy policy](#)[Terms of use](#)[Cookie preferences](#)

oscarrodriguezwork ▼